

E-Commerce Real-Time Stock Updates

Redis-backed multi-instance SSE flow and the single-server SSE flow, explained side by side.

What this PDF shows

1. Redis + SSE architecture
2. Without Redis (single server)
3. Where the message travels in each case

How to read it

The flow diagrams are left to right. The tables below highlight what each layer does and why Redis matters once you have more than one backend instance.

At a glance

- PostgreSQL is the source of truth for stock.
- Redis pub/sub shares stock change events across backend instances.
- SSE pushes the event from each backend instance to its connected browser tabs.
- Without Redis, the update stays inside one server instance.

1) Redis + SSE flow

This is the multi-instance version you just built. A checkout on any server updates the database, publishes a message to Redis, and every backend instance relays that message to its own SSE clients.

Redis-backed multi-instance flow



What each piece does

Layer	Job
PostgreSQL	Stores the real stock_count and transaction result
Redis pub/sub	Broadcasts the stock change to every backend instance
SSE	Keeps browser tabs connected to the local backend instance
React	Updates the product card or stock badge without page refresh

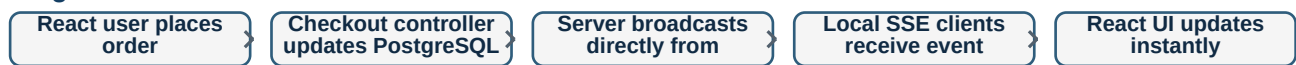
Typical Redis payload

```
{ type: 'stock_update', productId: 12, stock_count: 3, instock: true, changedBy: 'order', timestamp: '2026-04-21T10:30:00.000Z' }
```

2) SSE flow without Redis

This is the simple single-server version. It works because the order update and the SSE clients live on the same Node instance. The server can broadcast directly from memory without any Redis layer.

Single-server SSE flow



Why this works only on one server

Situation	Result
One backend instance	Works fine because SSE clients are stored in that same process
Two backend instances	Fails to sync across servers because each instance has its own in-memory client set
Redis added	Fixes this by sharing the event across all instances

Memory note

- In the no-Redis version, the client Set lives in RAM, so it disappears if the server restarts.
- In the Redis version, the event is not stored, only broadcast live to active subscribers.
- That is enough for real-time stock changes, because the database remains the source of truth.

3) Quick comparison

Aspect	Without Redis	With Redis
Works with 1 backend	Yes	Yes
Works with multiple backends	No	Yes
Stock source of truth	PostgreSQL	PostgreSQL
Event distribution	Local memory only	Redis pub/sub fan-out
Frontend transport	SSE	SSE

Recommended rule: update PostgreSQL first, commit the transaction, then publish the stock event to Redis. The UI should only react to the event after the database is already correct.

That gives you a clear story for both modes: single-server SSE for the simple case, and Redis + SSE for horizontal scaling.